

FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Angelo Morzenti

Exam of THURSDAY 4 JULY 2024 – Theory Part

WITH SOLUTIONS – FOR TEACHING PURPOSES THE SOLUTIONS ARE WIDELY COMMENTED

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

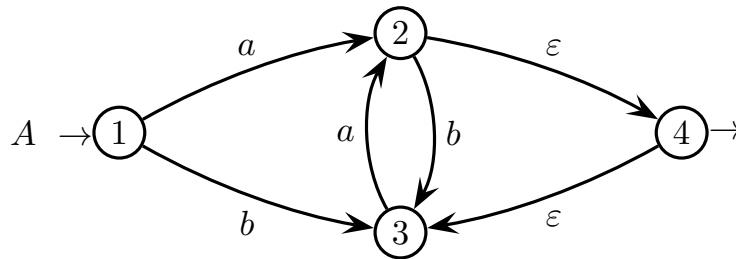
TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. A. MORZENTI

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): syntax and semantics of languages, divided in four sections:
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): compiler design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- For the theory part to be valid, the candidate must achieve a grade of at least 10/20 on each of the four sections.
- Correctly answering all the questions that are not marked optional, allows the candidate to achieve a high grade (although not the full mark).
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30.
- The final grade is the weighted average of the theory part (80 %) and of the lab part (20 %), and must be $\geq 18/30$.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: lab part 1 h – theory part 2 h.

1 Regular Expressions and Finite Automata 20%

1. Consider the nondeterministic automaton A below, over the two-letter alphabet $\Sigma = \{a, b\}$, with four states (initial state 1 and final state 4), which has spontaneous transitions (ε -transitions):



Answer the following questions:

- (a) Write all the valid strings of length ≤ 3 (less than or equal to three), and for those of length exactly three, list their computations.
 - (b) By applying the Berry-Sethi (BS) method to automaton A , find an equivalent deterministic automaton A_{det} . Examine if the deterministic automaton A_{det} is minimal or not, and if necessary minimize it.
 - (c) Cut all the spontaneous transitions from automaton A . Is the resulting automaton deterministic or not?
 - (d) By applying the Brzozowski-McCluskey (BMC) method to automaton A , find an equivalent regular expression R .
 - (e) (optional) Find a deterministic automaton $\overline{A}_{det}$ that recognizes the complement of language $L(A)$, and explain if such an automaton $\overline{A}_{det}$ is minimal or not.
-

Solution

(a) Here are the valid strings of length ≤ 3 (alphabetically sorted – six in total):

$a \quad a a \quad b a \quad a a a \quad a b a \quad b a a$

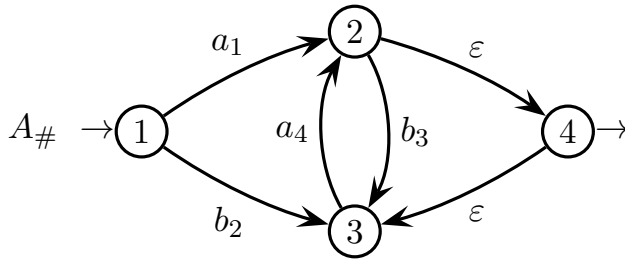
Here are the computations of the strings of length = 3 (three in total):

$\rightarrow 1 \xrightarrow{a} 2 \xrightarrow{\varepsilon} 4 \xrightarrow{\varepsilon} 3 \xrightarrow{a} 2 \xrightarrow{\varepsilon} 4 \xrightarrow{\varepsilon} 3 \xrightarrow{a} 2 \xrightarrow{\varepsilon} 4 \rightarrow$

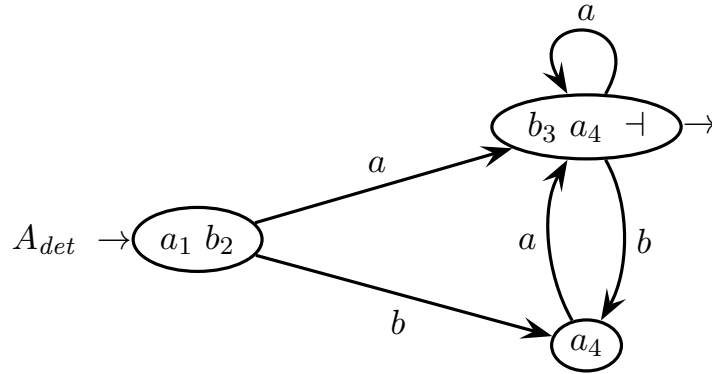
$\rightarrow 1 \xrightarrow{a} 2 \xrightarrow{b} 3 \xrightarrow{a} 2 \xrightarrow{\varepsilon} 4 \rightarrow$

$\rightarrow 1 \xrightarrow{b} 3 \xrightarrow{a} 2 \xrightarrow{\varepsilon} 4 \xrightarrow{\varepsilon} 3 \xrightarrow{a} 2 \xrightarrow{\varepsilon} 4 \rightarrow$

(b) Here is the Berry-Sethi (BS) method applied to automaton A :

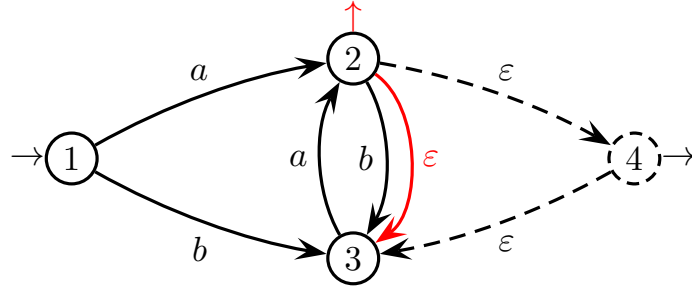


initials	$a_1 \ b_2$
terminal	followers
a_1	$b_3 \ a_4 \ \vdash$
b_2	a_4
b_3	a_4
a_4	$b_3 \ a_4 \ \vdash$

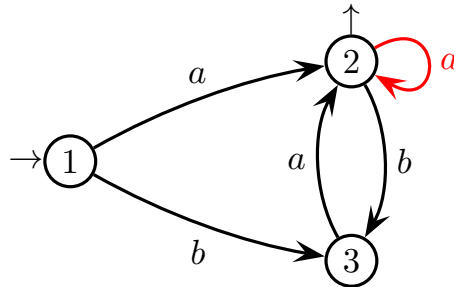


The deterministic automaton A_{det} is minimal. The two non-final states are distinguished by transition b , which is absent on state a_4 .

- (c) Cut transition $2 \xrightarrow{\varepsilon} 4$ by back-propagation. State 4 becomes useless, as well as transition $4 \xrightarrow{\varepsilon} 3$, thus both can be canceled (now state 2 is final), and a new spontaneous transition $2 \xrightarrow{\varepsilon} 3$ is introduced by back-propagation:

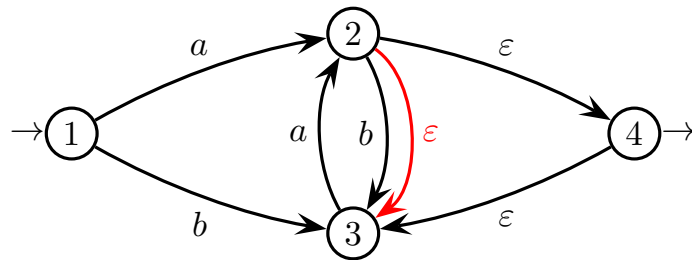


Cut transition $2 \xrightarrow{\varepsilon} 3$ by back-propagation:

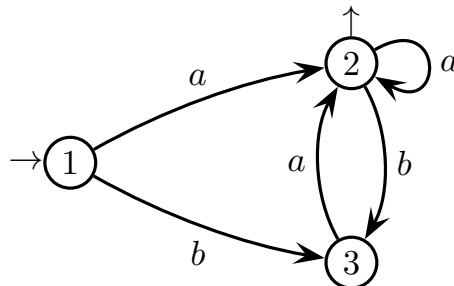


The resulting automaton is deterministic. Note it is identical to A_{det} .

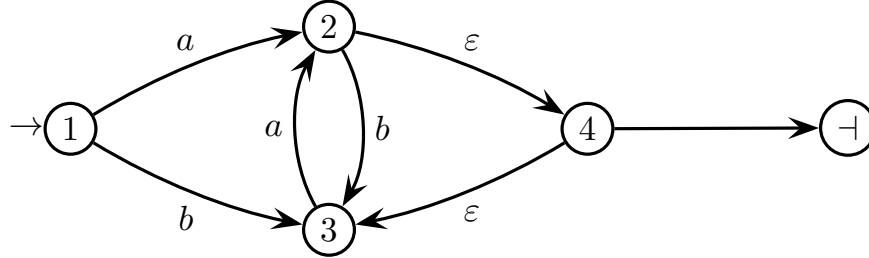
Alternatively, we can proceed (as explained in the textbook) by first taking on automaton A the transitive closure of the spontaneous transitions:



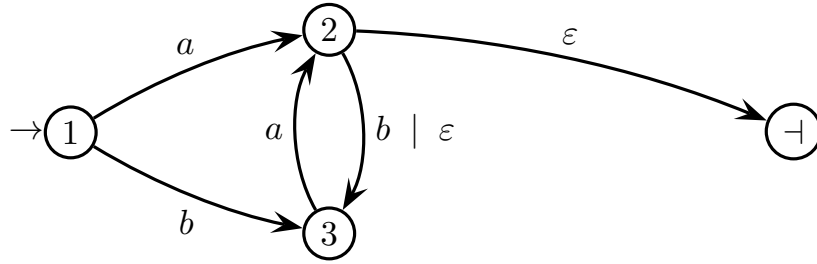
Then by retropropagating transition $3 \xrightarrow{a} 2$ through $2 \xrightarrow{\varepsilon} 3$, the transition $2 \xrightarrow{a} 2$ is introduced. By retropropagating the finality condition through $2 \xrightarrow{\varepsilon} 4$, state 2 becomes final. Finally all spontaneous transitions are removed, and state 4 is canceled because it is useless. The result is the same automaton as above:



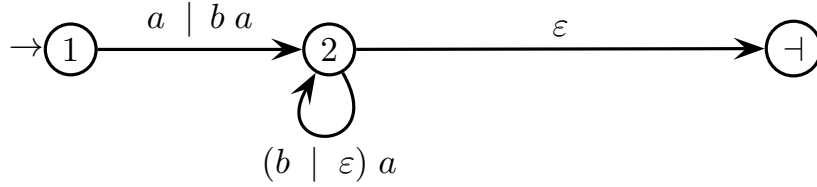
- (d) Here is the Brzozowski-McCluskey (BMC) method applied to automaton A . First normalize the automaton, as the final node 4 has an outgoing transition (no need of normalization for the initial node):



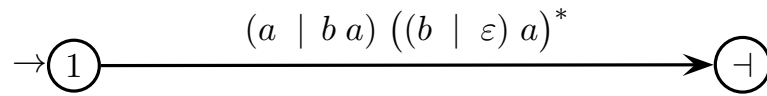
Eliminate node 4:



Eliminate node 3:



Eliminate node 2:

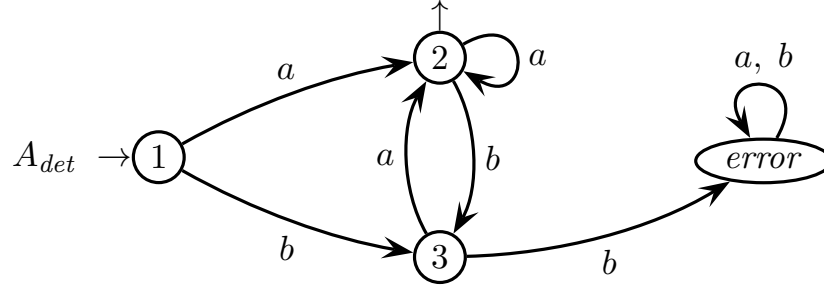


Therefore:

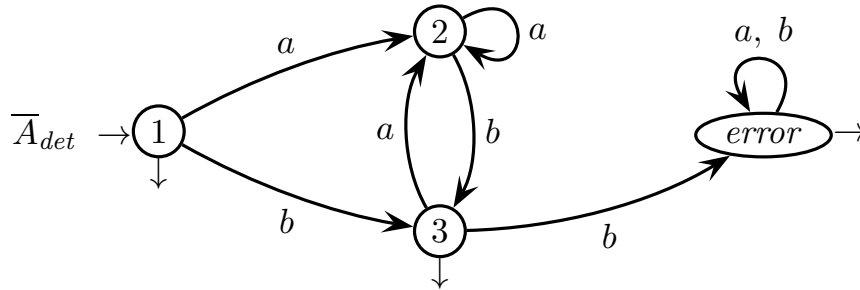
$$R = (a \mid b a) ((b \mid \varepsilon) a)^* = ([b] a) ([b] a)^* = ([b] a)^+$$

where the square brackets indicate optionality.

- (e) The complement can be straightforwardly obtained from the deterministic automaton A_{det} . Introduce the error state to complete the automaton:



Here is the complement automaton $\overline{A}_{det}$:



Notice that the complement automaton is minimal, too. This is easily seen from the distinguishability relation:

2	×		
3	(2, 2) (3, <i>error</i>)	×	
<i>error</i>	×	×	<i>×</i>
	1	2	3

Most state pairs are immediately distinguishable. State 2 is non-final, while all the others, i.e., 1, 3 and *error*, are final and are therefore distinguishable from 2. Both states 1 and 3 go by input *a* to a non-final state (state 2), while state *error* goes by input *a* to a final one (itself), thus *error* is distinguishable from both 1 and 3. At this point, we are only left with deciding about pair (1, 3), but it turns out that states 1 and 3, which respectively go by input *b* to states 3 and *error*, are also distinguishable since their destinations 3 and *error* are so (from what said before – see red above). All the states are therefore distinguishable.

2 Free Grammars and Pushdown Automata 20%

1. Consider the two languages L_1 and L_2 defined below, over the two-letter alphabet $\Sigma = \{ a, b \}$:

$$L_1 = \{ a^{2n} b^{n+k} \mid n > 0 \wedge k \geq 0 \}$$

$$L_2 = \{ a^{2n} b^k \mid n \geq 0 \wedge 0 \leq k \leq 2n \}$$

Answer the following questions:

- (a) With reference to the language L_1 , define a non-ambiguous BNF grammar G_1 that generates it and draw the syntax tree of the sample string:

$$a^2 b^3$$

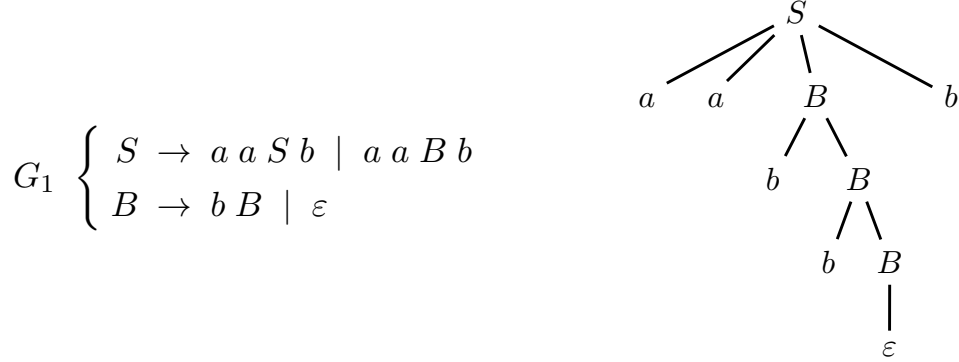
- (b) With reference to the language L_2 , define a non-ambiguous BNF grammar G_2 that generates it and draw the syntax tree of the sample string:

$$a^6 b^3$$

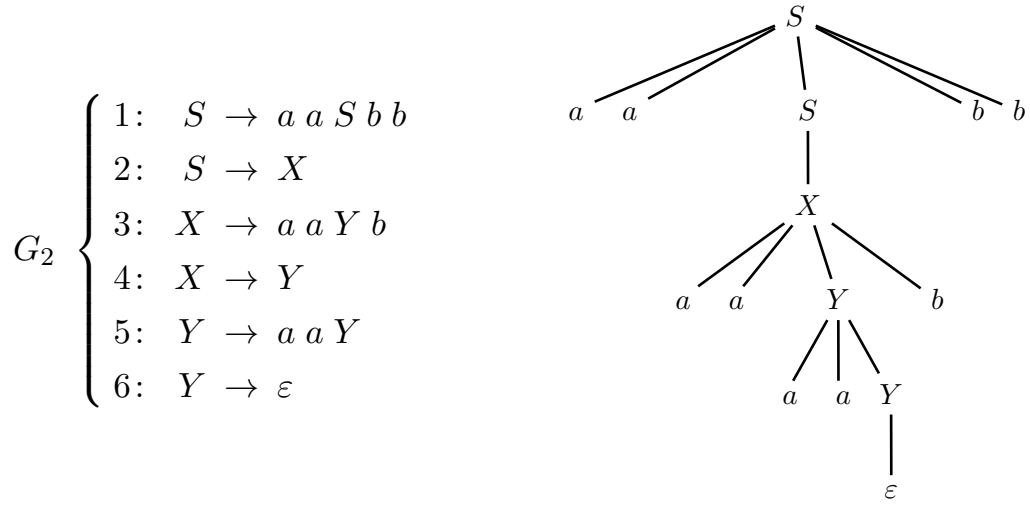
- (c) (optional) Explain why grammar G_2 is not ambiguous (avoid generic arguments, and refer explicitly to the features of G_2).
-

Solution

- (a) Here is a viable non-ambiguous BNF grammar G_1 for language L_1 (axiom S) and the syntax tree in G_1 of the sample string $a^2 b^3$:



- (b) Here is a viable non-ambiguous BNF grammar G_2 for language L_2 (axiom S), with the rules assigned an increasing integer index, and the syntax tree in G_2 of the sample string $a^6 b^3 = a^{2n} b^k$ with $n = 3$ and $k = 3$:



- (c) Grammar G_2 is not ambiguous for the reasons below. To generate a string $a^{2n} b^k$:
- rule 1 is applied for a number of times s equal to $\lfloor k/2 \rfloor$ — where the rounding operator $\lfloor x \rfloor$ (called “Gauss parenthesis”) indicates the greatest integer $\leq x$ — as there is no other way to produce a number of b characters ≥ 2
 - rule 3 is applied for a number of times t equal to 1 or 0 if k is odd or even, respectively, as there is no other rule that produces an odd number of b characters
 - rule 5 is applied for $n - s - t$ times as there is no other way to produce a characters not paired with any b

Furthermore, the produce relation among nonterminals is $S \rightarrow X \rightarrow Y$. Therefore for every string in the language its derivation in grammar G_2 has a unique sequence of rule applications.

2. A programming language, which can be seen as a simplified version of TypeScript, allows the definition of inductive custom data types such as:

```
type Tree = number | string | Tree [ ]
```

where symbol **Tree** is the identifier of the type to be declared, and symbols **number** and **string** are terminals that denote primitive types. Apart from **number** and **string**, one more primitive type is available: **boolean**. The notation **Tree []** denotes an *array* of elements of type **Tree**. The expression **number | string | Tree []** denotes a *union* non-primitive type where, in this case, the objects of that type can be numbers, strings or arrays of **Tree**. In addition to union and array, the language also allows a *tuple* type, which is constructed as in the following example:

```
type Student = [ string, string, number, number, boolean ]
```

where, e.g., the type **Student** is defined by their surname and name, two strings denoted by the primitive type **string**, their person code and enrollment code, both denoted by the primitive type **number**, and whether they have a currently active career, denoted by the primitive type **boolean**.

Answer the following questions:

- (a) Define a non-ambiguous EBNF grammar for the type definition statement. In the grammar, identifiers shall be denoted by the nonterminal $\langle \text{ID} \rangle$, which is assumed to be already defined elsewhere and does not need to be expanded.
- (b) Draw the parse tree of the sentence below:

```
type Tree = number | Tree [ ]
```

- (c) (optional) Extend the grammar to include the *object* constructed type, which is similar to a tuple, but allows each element to be named, and possibly optional, as in the following example:

```
type Student = { surname: string,
                  name: string,
                  person_code : number,
                  enrollment_code ? : number,
                  active : boolean
                }
```

where the terminal **?** denotes the optionality of the enrollment code field. Field names are modeled as identifiers $\langle \text{ID} \rangle$.

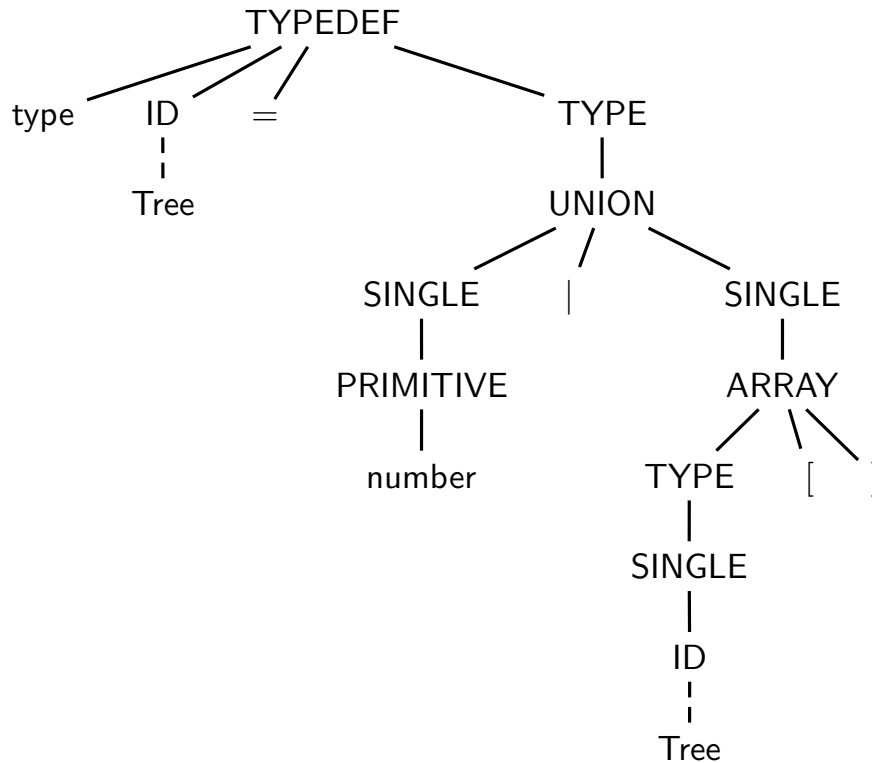
Solution

- (a) Here is a viable non-ambiguous EBNF grammar (axiom $\langle \text{TYPEDEF} \rangle$). The non-terminal $\langle \text{ID} \rangle$ does not need to be expanded:

$$\left\{ \begin{array}{l} \langle \text{TYPEDEF} \rangle \rightarrow \text{type } \langle \text{ID} \rangle \text{ "=" } \langle \text{TYPE} \rangle \\ \langle \text{TYPE} \rangle \rightarrow \langle \text{SINGLE} \rangle \mid \langle \text{UNION} \rangle \\ \langle \text{SINGLE} \rangle \rightarrow \langle \text{PRIMITIVE} \rangle \mid \langle \text{ARRAY} \rangle \mid \langle \text{TUPLE} \rangle \mid \langle \text{ID} \rangle \\ \langle \text{UNION} \rangle \rightarrow \langle \text{SINGLE} \rangle (\text{"|"} \langle \text{SINGLE} \rangle)^+ \\ \langle \text{PRIMITIVE} \rangle \rightarrow \text{number} \mid \text{string} \mid \text{boolean} \\ \langle \text{ARRAY} \rangle \rightarrow \langle \text{TYPE} \rangle \text{"["} \text{" " } \\ \langle \text{TUPLE} \rangle \rightarrow \text{"["} \langle \text{TYPE} \rangle (\text{" ," } \langle \text{TYPE} \rangle)^* \text{" " } \end{array} \right.$$

Union ($\langle \text{UNION} \rangle$) and non-union ($\langle \text{SINGLE} \rangle$) types are described in separate rules to allow the $\langle \text{UNION} \rangle$ rule to be unambiguous.

- (b) Here is the syntax tree of the sample phrase `type Tree = number | Tree [ ]`:



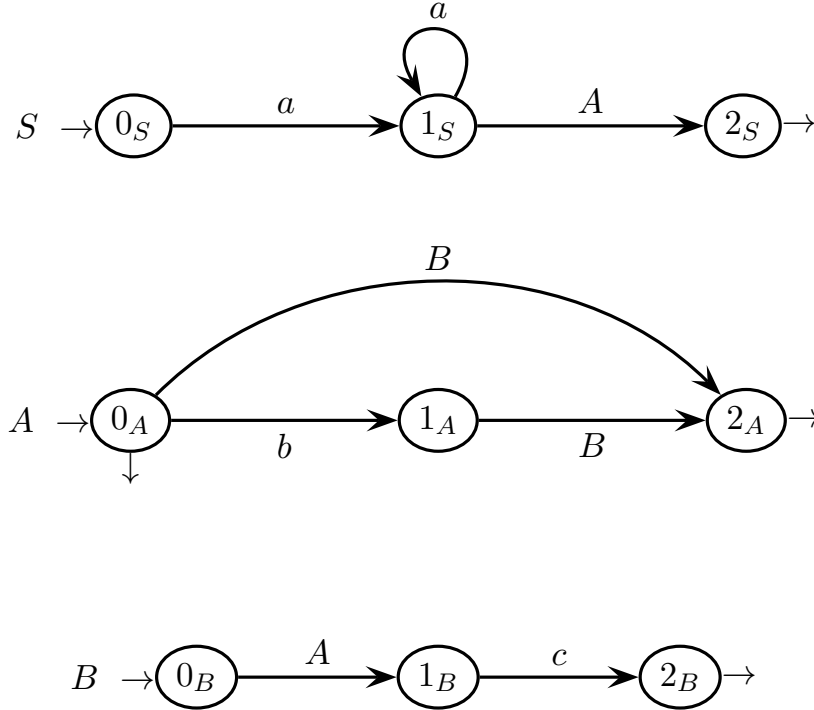
- (c) Here is the grammar extended with objects. The nonterminals **OBJECT** and **OBJFLD** are added, with their rules:

$$\left\{ \begin{array}{l} \langle \text{SINGLE} \rangle \rightarrow \langle \text{PRIMITIVE} \rangle \mid \langle \text{ARRAY} \rangle \mid \langle \text{TUPLE} \rangle \mid \langle \text{ID} \rangle \mid \langle \text{OBJECT} \rangle \\ \langle \text{OBJECT} \rangle \rightarrow \text{"{" } \langle \text{OBJFLD} \rangle (\text{" ," } \langle \text{OBJFLD} \rangle)^* \text{"}" } \\ \langle \text{OBJFLD} \rangle \rightarrow \langle \text{ID} \rangle [\text{"?" }] \text{" : " } \langle \text{TYPE} \rangle \end{array} \right.$$

The changes only are shown (in red). Square brackets indicate optionality.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the grammar G below, represented by a machine net with the three-symbol nonterminal alphabet $V = \{S, A, B\}$ (axiom S) and the three-letter terminal alphabet $\Sigma = \{a, b, c\}$:



Answer the following questions (use the figures / tables / spaces on the next pages):

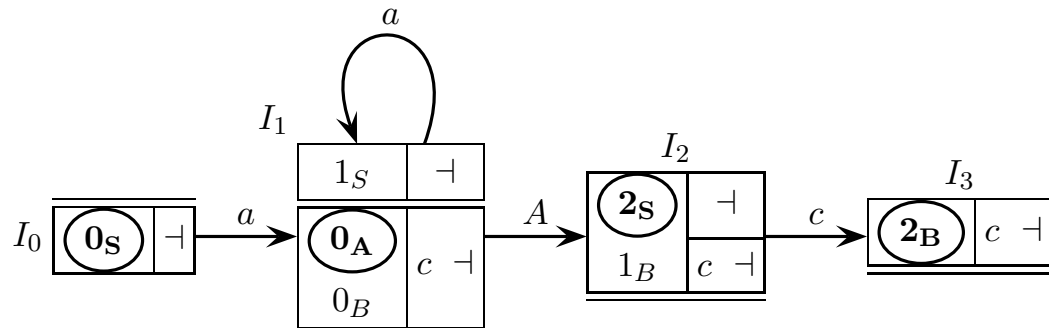
- (a) Write all the valid strings of grammar G of length ≤ 3 (less than or equal to three), and draw the syntax trees of the strings of length exactly three.
- (b) Draw the complete pilot of grammar G (part of it is already given) and say if grammar G is of type ELR. Explain your answer. Does the pilot have multiple transitions, and if so, which ones?
- (c) Find all the guide sets on the net of grammar G and prove that grammar G is **not** ELL(k), for any $k \geq 1$. Explain your answer.
- (d) (optional) Find a possible formulation of language $L(G)$ as a set of strings, like:

$$L(G) = \{ \text{string model(s)} \mid \text{characteristic predicate} \}$$

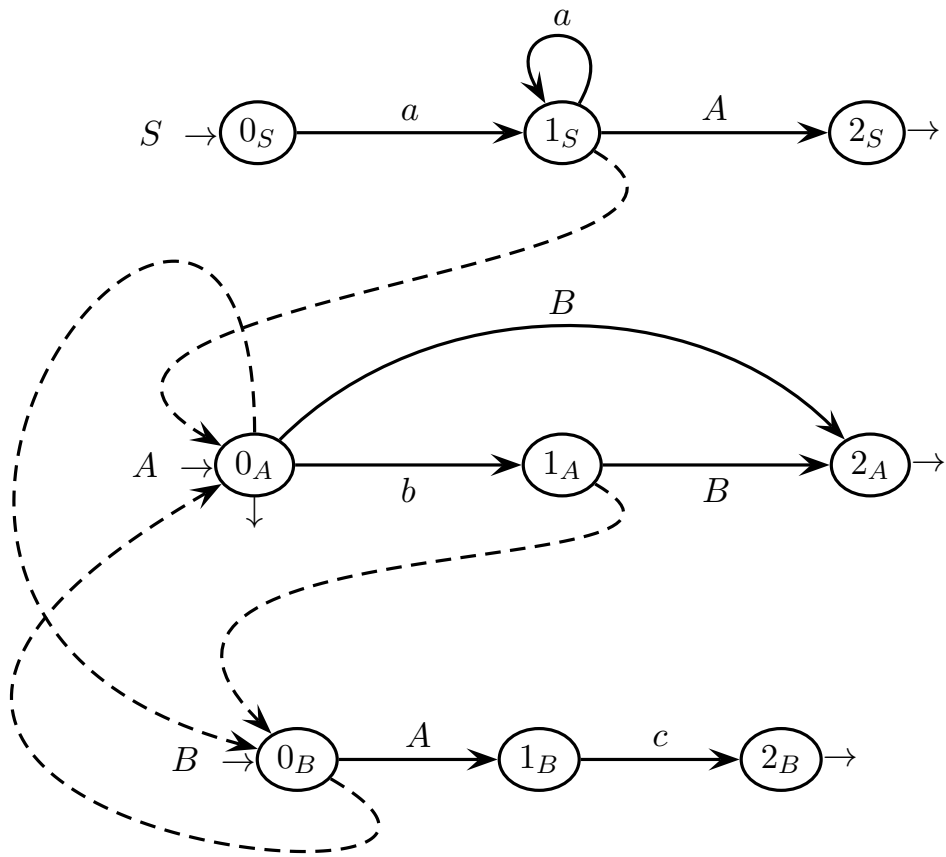
Say if language $L(G)$ is regular or not, and explain your answer.

here write the strings and draw the trees – question (a)

draw here the complete pilot and examine ELR – question (b)



write here the guide sets and examine ELL – question (c)



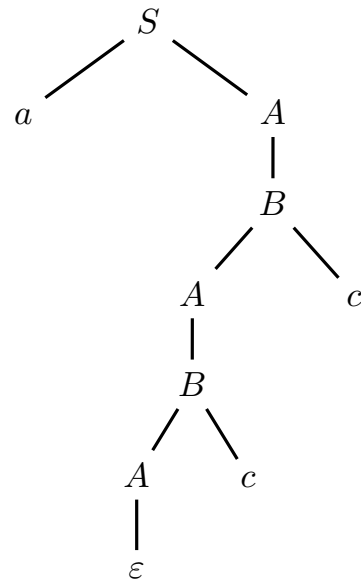
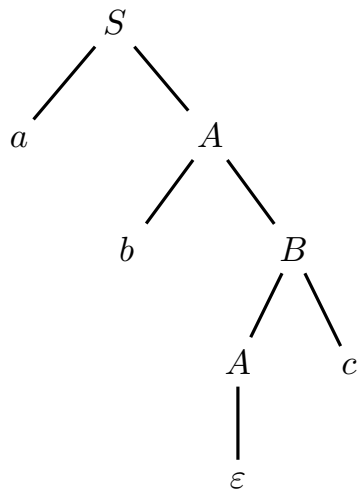
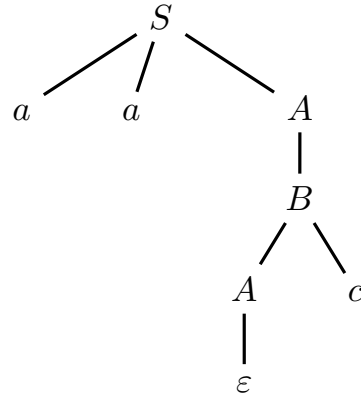
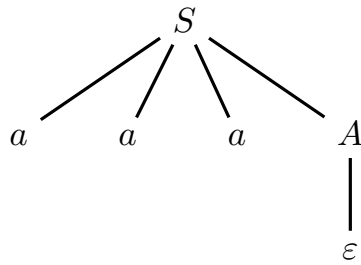
space for answering question (d)

Solution

(a) Strings (alphabetically sorted – seven in total):

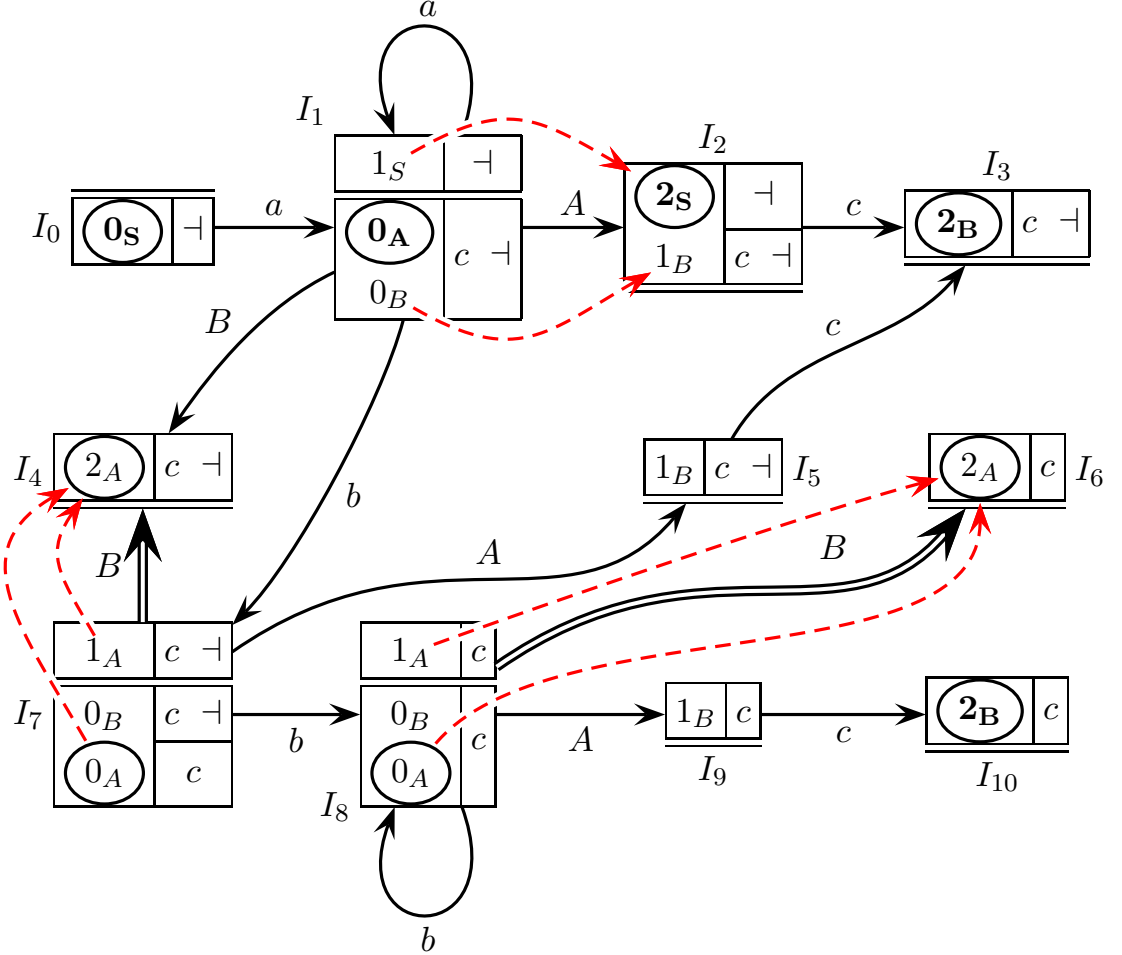
$a \quad aa \quad ac \quad aaa \quad aac \quad abc \quad acc$

Trees of the strings of length = 3 (four in total):



Note that sooner or later the rule $A \rightarrow \varepsilon$ has to be used to terminate the left recursion.

(b) Pilot, it has 11 states (the states from 0 to 3 are already given):

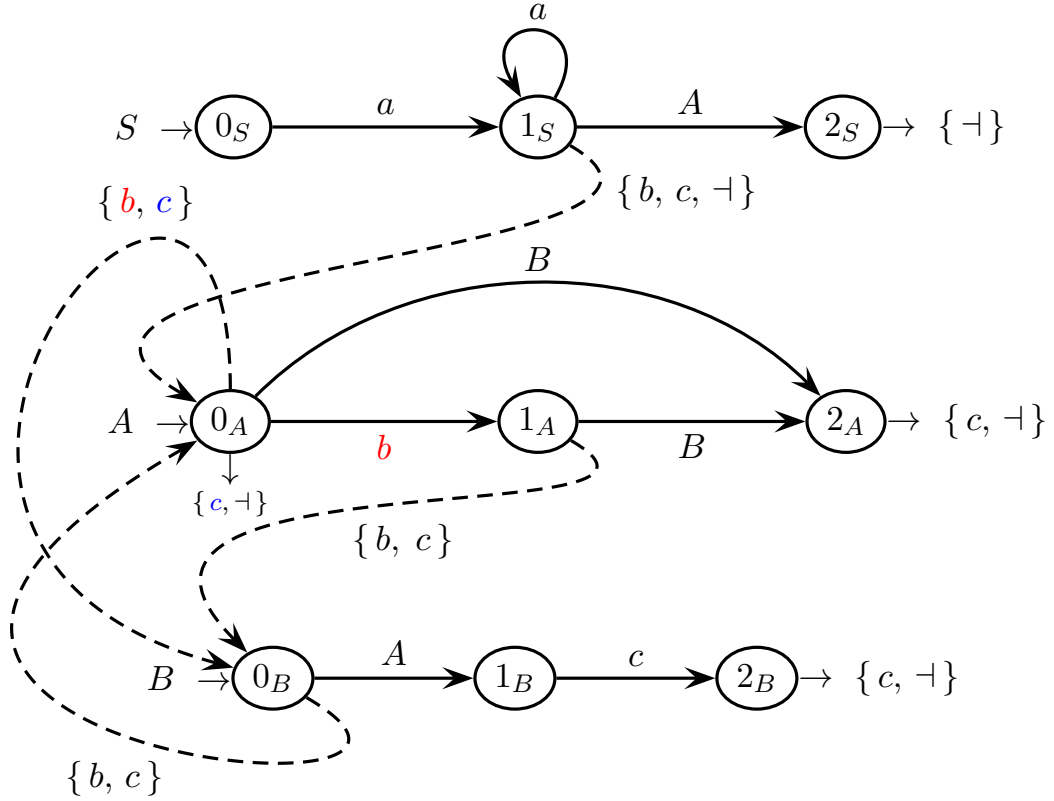


There are conflicts, in particular of the Convergence type (while conflicts of SR or RR type are absent). The transitions $I_7 \xrightarrow{B} I_4$ and $I_8 \xrightarrow{B} I_6$ of the pilot are double, convergent and conflicting on terminal c (see the dashed red arrows). Thus grammar G is not ELR.

Actually, in the machine net we can observe that there are two B -transitions, i.e., $0_A \xrightarrow{B} 2_A$ and $1_A \xrightarrow{B} 2_A$, converging on state 2_A , which is a necessary (though not sufficient) condition for convergence to exist in the pilot. This observation suggests to examine closely the pilot, to check whether convergence really exists involving some B -transitions. In the present case the pilot does exhibit convergence, and even conflicting, as said before.

Note also that the pilot does not have the STP, as transition $I_1 \xrightarrow{A} I_2$ is also double, but not convergent (see the dashed red arrows). Transition $I_1 \xrightarrow{B} I_4$ is instead single, though it is labeled by B like the other two converging B -transitions.

(c) Guide sets (for $k = 1$):



The bifurcation states are 1_S and 0_A . State 0_A has conflicts on b and c , so grammar G is not ELL for $k = 1$. Grammar G is left-recursive (not immediately), e.g., the derivation $A \Rightarrow B \Rightarrow A c$, thus it is not ELL(k) for any $k \geq 1$.

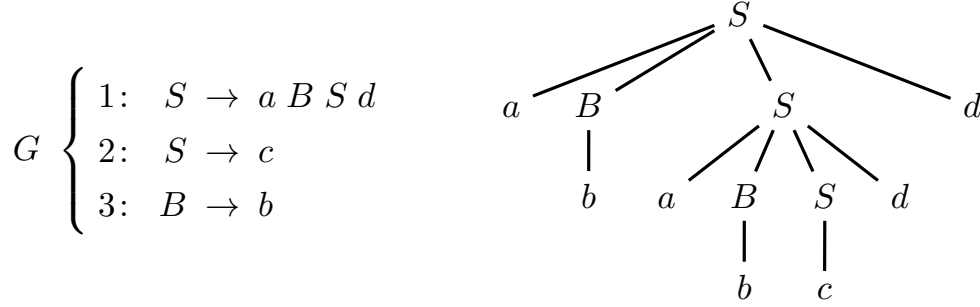
- (d) Of course, in grammar G there is a derivation $S \Rightarrow a^+ A \Rightarrow a^+ b B \Rightarrow a^+ b A c \Rightarrow \dots \Rightarrow a^+ b^h A c^h$, so the number of letters c is at least equal to that of letters b . However, since derivation $A \Rightarrow B$ is also possible, which does not generate letter b , the number of letters c may be greater. The number of letters a is instead independent. Here is a viable language formulation:

$$L(G) = \{ a^i b^h c^k \mid i \geq 1 \wedge 0 \leq h \leq k \}$$

It is well-known that the relationship $h \leq k$ between exponents prevents the language from being regular, as no regular expression can generate only strings that satisfy such a numeric inequality.

4 Language Translation and Semantic Analysis 20%

1. Consider the BNF grammar G below (left), with nonterminals S and B (axiom S), and terminals $\{a, b, c, d\}$, where the rules are numbered for reference:



The string $ababcbdd \in L(G)$ has the syntax tree shown above (right). It admits the leftmost derivation below (we subscript to the derivation symbol $\Rightarrow$ the number of the applied rule):

$$S \xRightarrow{1} a B S d \xRightarrow{3} a b S d \xRightarrow{1} a b a B S d d \xRightarrow{3} a b a b S d d \xRightarrow{2} a b a b c d d$$

and the rightmost derivation below:

$$S \xRightarrow{1} a B S d \xRightarrow{1} a B a B S d d \xRightarrow{2} a B a B c d d \xRightarrow{3} a B a b c d d \xRightarrow{3} a b a b c d d$$

Answer the following questions:

- (a) We intend to define a translation function $\tau_1: L(G) \rightarrow \{1, 2, 3\}^+$ that, for every string $x \in L(G)$, returns the *leftmost* derivation of x , encoded as the sequence of the rules applied to derive x . For instance:

$$\tau_1(ababcbdd) = 13132$$

Write a translation grammar or scheme G_1^T that computes translation τ_1 , with source grammar G , and draw the translation tree for the string $ababcbdd$.

- (b) We intend to define a translation function $\tau_2: L(G) \rightarrow \{1, 2, 3\}^+$ that, for every string $x \in L(G)$, returns the *rightmost derivation in reverse order* of x . For instance:

$$\tau_2(ababcbdd) = 33211$$

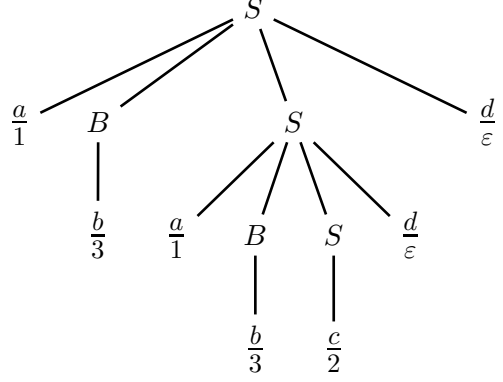
Write a translation grammar or scheme G_2^T that computes translation τ_2 , with source grammar G , and draw the translation tree for the string $ababcbdd$.

- (c) Is translation τ_1 invertible? Explain your answer.
- (d) Is translation τ_2 invertible? Explain your answer.
- (e) (optional) Is translation τ_1 deterministic? Explain your answer.
- (f) (optional) Is translation τ_2 deterministic? Explain your answer.

Solution

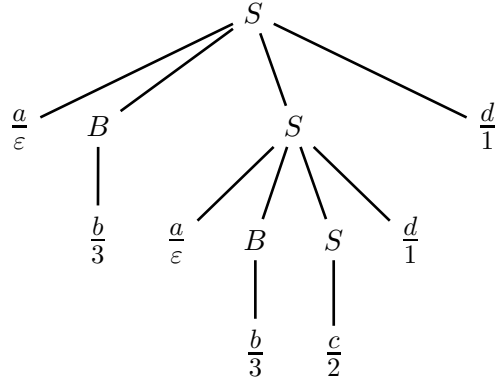
- (a) Here is a viable translation grammar G_1^τ (axiom S) and the translation tree of the sample string $ababccd$:

$$G_1^\tau \left\{ \begin{array}{l} 1: S \rightarrow \frac{a}{1} B S \frac{d}{\varepsilon} \\ 2: S \rightarrow \frac{c}{2} \\ 3: B \rightarrow \frac{b}{3} \end{array} \right.$$



- (b) Here is a viable translation grammar G_2^τ (axiom S) and the translation tree of the sample string $ababccd$:

$$G_2^\tau \left\{ \begin{array}{l} 1: S \rightarrow \frac{a}{\varepsilon} B S \frac{d}{1} \\ 2: S \rightarrow \frac{c}{2} \\ 3: B \rightarrow \frac{b}{3} \end{array} \right.$$



- (c) Translation τ_1 is invertible, because since the grammar $G_1^s = G$ is non-ambiguous (it is obviously ELL(1)), every string of language $L(G)$ is uniquely identified by its leftmost derivation.
- (d) Translation τ_2 is also invertible, because since the grammar $G_2^s = G$ is non-ambiguous, every string of language $L(G)$ is also uniquely identified by its rightmost derivation.
- (e) Translation τ_1 is deterministic, because it is functionally equivalent to a top-down parser, and grammar G_1^s is ELL(1).
- (f) Translation τ_2 is deterministic, because it is functionally equivalent to a bottom-up parser, and grammar G_2^s , being ELL(1), is also ELR(1).

2. Consider the syntactic support G shown below, over the terminal alphabet $\Sigma = \{v, "\oplus", ":", "=", n\}$ and nonterminal alphabet $V = \{S, X, A, T\}$, defined by these rules (axiom S):

$$G \left\{ \begin{array}{ll} 1: & S \rightarrow X \\ 2: & X \rightarrow A ":" X \\ 3: & X \rightarrow \varepsilon \\ 4: & A \rightarrow v "=" T "\oplus" T \quad // \text{ assignment} \\ 5: & T \rightarrow v \\ 6: & T \rightarrow n \end{array} \right.$$

Grammar G represents a simple programming language, where a program is a sequence (X) of assignments (A). Variables are represented by terminal v . Two functions $f(v)$ and $l(t)$ are available to get the name of variable v and the line number corresponding to each terminal t , respectively. Numeric constants and arithmetic operators are represented by terminals n and $\oplus$, respectively.

We want to define an attribute grammar over the support G to enable the issuing of semantic warnings related to the use of variables that are not yet defined. To this end, we define the following attributes s , d and e :

- s is a right attribute inherited by the nonterminals X , A and T , which represents the set of variables defined as far as that point in the program; the value of s is a set of variable names
- d is a left attribute synthesized by nonterminal A , which represents the variable defined by the current assignment; the value of d is a set that contains a single variable name.
- e is a left attribute synthesized by all the nonterminals (S , X , A and T), which represents the detected errors, i.e., the lines containing utilizations of variables that are not previously defined; the value of e is a set of line numbers

Answer the following questions (use the tables / trees / spaces on the next pages):

- (a) Write the semantic functions for the three attributes s , d and e above.
- (b) Write the computed values of the three attributes for the program below, which consists of three assignments:

```
a = 1 + 3;    // line 1
d = 2 + a;    // line 2  NB: variable a was previously defined
b = c + d;    // line 3  NB: d was previously defined, but not c
```

and decorate the parse tree of the program (see the next pages). The line numbers are not part of the program, but provide the values for function l .

- (c) Prove that the attribute grammar is $LL(k)$ (for some $k \geq 1$) and one-sweep.
- (d) Write the semantic procedure (in pseudocode) for analyzing nonterminal A .

table to be filled with the semantic functions – question (a)

#	<i>syntax</i>	<i>semantic functions</i>
1:	$S_0 \rightarrow X_1$	$s_1 :=$ $e_0 :=$
2:	$X_0 \rightarrow A_1 \text{ “;” } X_2$	$s_1 :=$ $s_2 :=$ $e_0 :=$
3:	$X_0 \rightarrow \varepsilon$	$e_0 :=$
// nonterminal A generates an assignment – it is associated to the attributes s , d and e // the attribute values of s , d and e for A must be shown in the program of question (b)		
4:	$A_0 \rightarrow v \text{ “=” } T_1 \text{ “}\oplus\text{” } T_2$	$s_1 :=$ $s_2 :=$ $d_0 :=$ $e_0 :=$
5:	$T_0 \rightarrow v$	$e_0 :=$
6:	$T_0 \rightarrow n$	$e_0 :=$

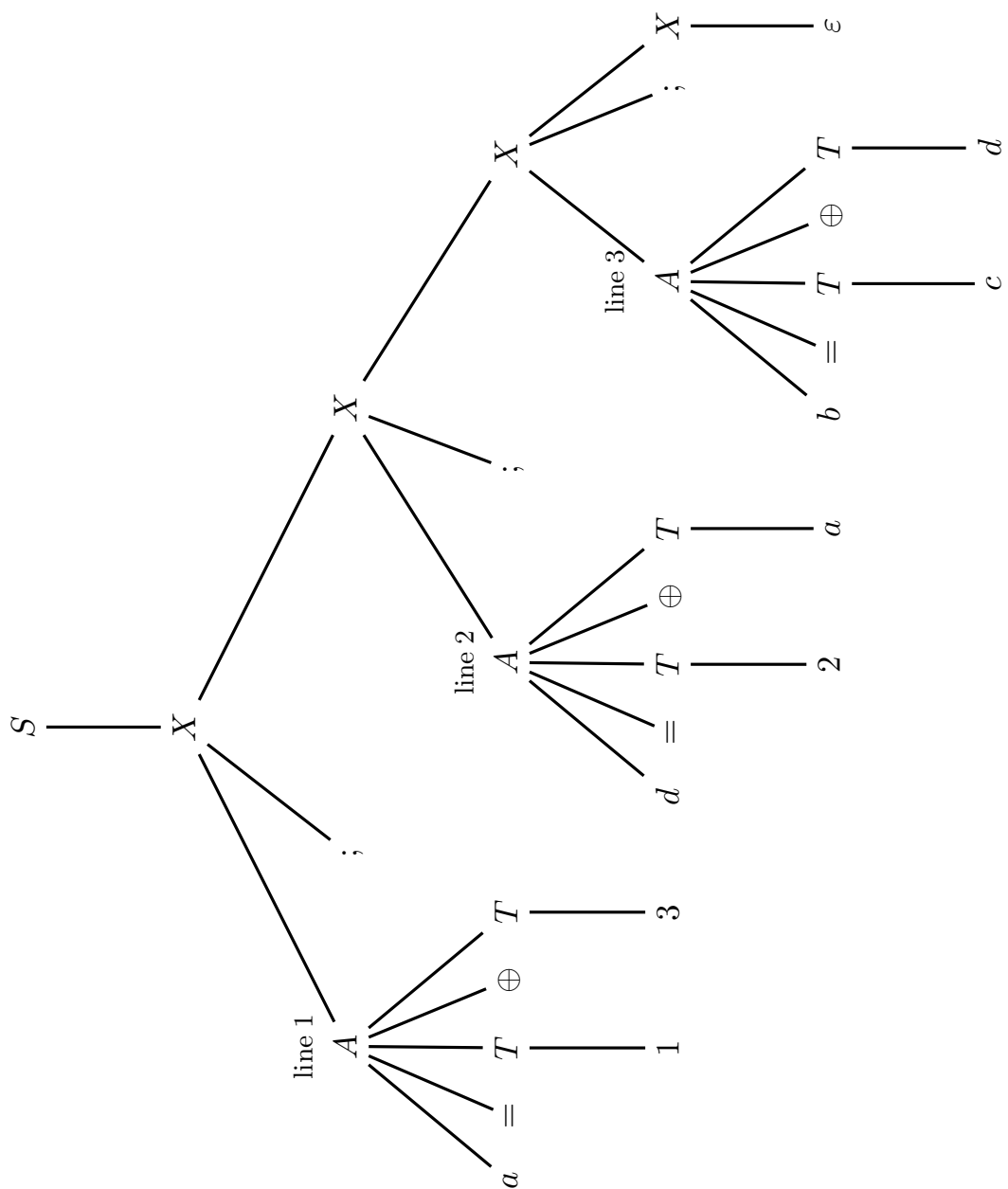
program and parse tree to be decorated with attribute values – question (b)

// write here the attribute values s0, d0, e0 for each assignment

a = 1 + 3; // line 1 s = d = e =

d = 2 + a; // line 2 s = d = e =

b = c + d; // line 3 s = d = e =



space for answering question (c)

space for answering question (d)

Solution

- (a) Here are the semantic functions for the three attributes s , d and e :

#	<i>syntax</i>	<i>semantic functions</i>
1:	$S_0 \rightarrow X_1$	$s_1 := \emptyset$ // initialization $e_0 := e_1$ // collect errors
2:	$X_0 \rightarrow A_1 \text{ “;” } X_2$	$s_1 := s_0$ // propagate declarations $s_2 := s_0 \cup d_1$ // add decl. & propag. $e_0 := e_1 \cup e_2$ // collect errors
3:	$X_0 \rightarrow \varepsilon$	$e_0 := \emptyset$ // initialization
// attribute values of s , d and e for A are listed in the program at point (b)		
4:	$A_0 \rightarrow v \text{ “=” } T_1 \text{ “} \oplus \text{” } T_2$	$s_1 := s_0$ // propagate declarations $s_2 := s_0$ // propagate declarations $d_0 := \{f(v)\}$ // initialization $e_0 := e_1 \cup e_2$ // collect errors
5:	$T_0 \rightarrow v$	if $f(v) \in s_0$ then // verification $e_0 := \emptyset$ // initialization: correct else $e_0 := \{l(v)\}$ // initialization: error endif
6:	$T_0 \rightarrow n$	$e_0 := \emptyset$ // initialization

The attribute grammar is acyclic, thus it is computable (with an appropriate visit order of the tree nodes).

- (b) Here are the attribute values for the program, and the decorated parse tree:

// attribute values s_0 , d_0 , e_0 for each assignment (nonterm. A)

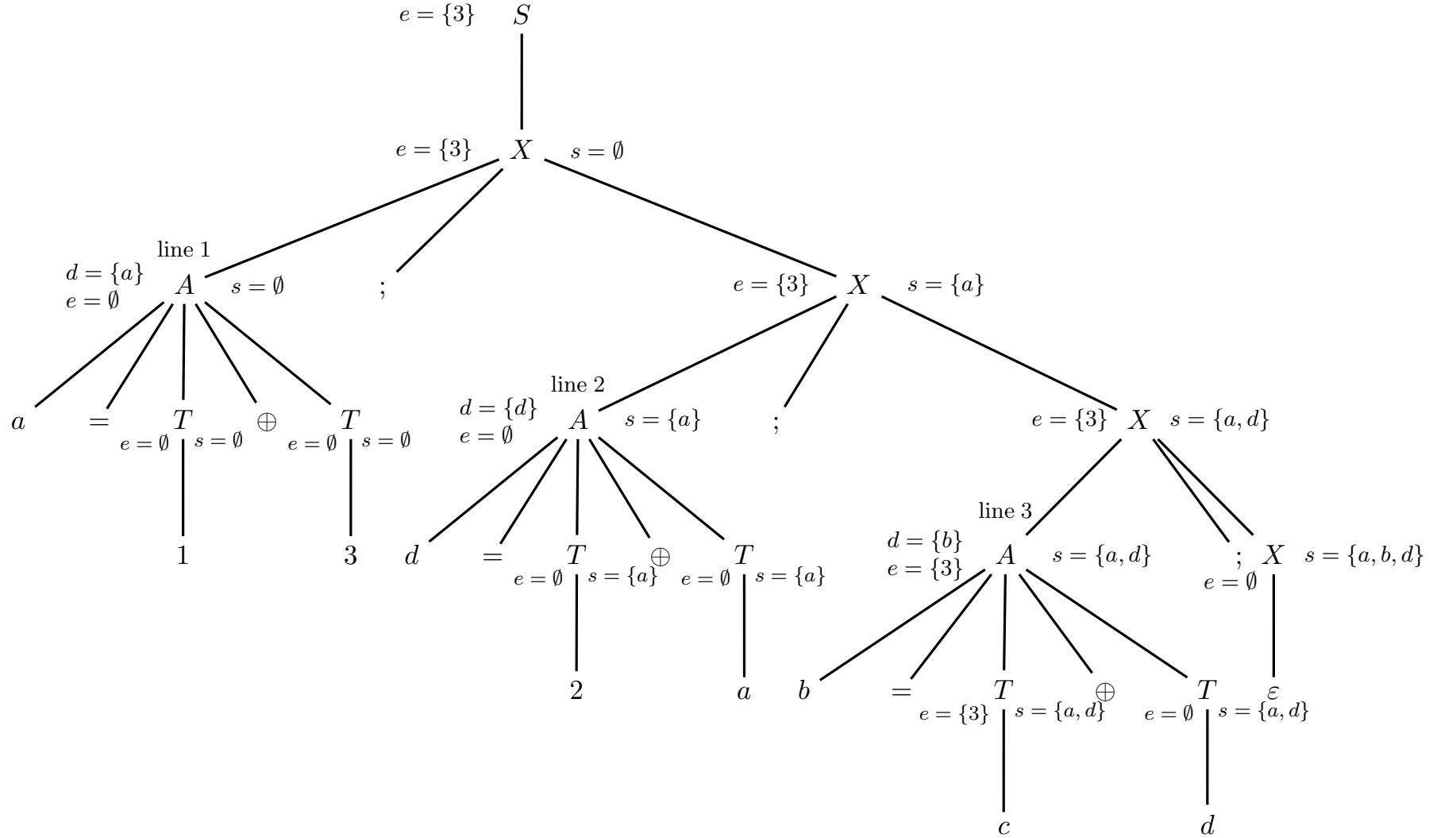
$a = 1 + 3;$ // line 1 $s = \{ \}$ $d = \{a\}$ $e = \{ \}$

$d = 2 + a;$ // line 2 $s = \{a\}$ $d = \{d\}$ $e = \{ \}$

$b = c + d;$ // line 3 $s = \{a, d\}$ $d = \{b\}$ $e = \{3\}$

The assignment at line 3 is incorrect, because variable c is used even though it is not declared in any of the previous statements at lines 1 and 2. Therefore the attribute e of statement 3 contains line number 3. The attributes e of the other two statements are empty, because such statements are correct. It is easy to assign a value to the attributes s and d , according to their definitions: s contains the previously defined variables and d contains the currently defined one.

decorated parse tree – the attribute values of the nodes A are shown in the program above



- (c) The syntactic support G is of type LL(1). The alternatives 2 and 3 of nonterminal X have look-ahead v and $\neg$, respectively. The alternatives 5 and 6 of nonterminal T have look-ahead v and n , respectively. Since the alternative look-ahead symbols are different, the support is LL(k) for $k = 1$.

The attribute grammar is one-sweep. Attributes d and e are synthesized: attribute e depends only on itself and on an external function, and attribute d is derived from a terminal through a function. Attribute s is inherited: it depends only on itself, except for the rule 2 where the attribute instance s_2 of nonterminal X_2 depends on attribute d , which is synthesized in the sibling node of nonterminal A_1 . Thus it suffices to assume that in the rule 2 the subtree of A is visited before the subtree of X ; said differently, the sibling graph of rule 2 is acyclic (and the sibling graph of rule 4 is even empty). In conclusion the attribute grammar is computable with a top-down bottom-up tree visit, so it is one-sweep.

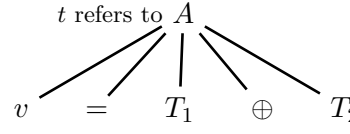
Finally, the attribute grammar satisfies the L-condition, since it is LL and one-sweep, and in the rule 2 the semantic visit order of the subtrees of A and X is first A_1 then X_2 (as said before), i.e., is coincident with the syntactic visit order. Thus we can design an integrated syntactic-semantic recursive-descent analyzer.

- (d) Here is the pseudocode of the semantic procedure for nonterminal A (see on the left), which has the single rule 4: $A \rightarrow v \text{ "=" } T \text{ "}\oplus\text{" } T$:

```

procedure  $A$  (in:  $t, s$ ; out:  $d, e$ )
  var  $s_1, s_2, e_1, e_2$ 
  // rule  $A \rightarrow v = T_1 \oplus T_2$ 
  // ptr  $t$  refers to node  $A$ 
   $s_1 := s$ 
  call  $T$  ( $t \rightarrow T_1, s_1; e_1$ )
   $s_2 := s$ 
  call  $T$  ( $t \rightarrow T_2, s_2; e_2$ )
   $d := f(t \rightarrow v)$ 
   $e := e_1 \cup e_2$ 
  return
end procedure

```

t refers to A

 $v \quad = \quad T_1 \quad \oplus \quad T_2$

```

// optimized version
procedure  $A$  (in:  $t, s$ ; out:  $d, e$ )
  var  $e_1, e_2$ 
  call  $T$  ( $t \rightarrow T_1, s; e_1$ )
  call  $T$  ( $t \rightarrow T_2, s; e_2$ )
   $d := f(t \rightarrow v)$ 
   $e := e_1 \cup e_2$ 
  return
end procedure

```

It is assumed that the complete syntax tree has been pre-built through previous syntax analysis. The input parameter t is the pointer to the syntax (sub)tree corresponding to nonterminal A (see the elementary tree above corresponding to the rule 4 of A). The subtrees T_1 and T_2 , and the assigned variable v , are child nodes of node A , thus in the procedure they are referred to as $t \rightarrow T_1$, $t \rightarrow T_2$ and $t \rightarrow v$, through the pointer t to their parent node A .

Some coding optimization is possible (see on the right). Since the local (attribute) variables s_1 and s_2 are a copy of the input (inherited) attribute s , they can be simply replaced by s . Instead, both local (attribute) variables e_1 and e_2 are necessary, as they contribute in different ways to the computation of the output (synthesized) attribute e . The output (synthesized) attribute d is just a function of a terminal.

For completeness, here is also the pseudocode of the integrated syntactic-semantic procedure (for brevity with both syntactic and semantic optimization):

```

// integrated and optimized version
procedure  $A$  (in:  $s$ ; out:  $d, e$ )
var  $e_1, e_2$ 
if  $cc \in \{v\}$  then
     $d := f(cc)$  // here the token  $cc$  contains variable  $v$ 
     $cc := next$ 
    if  $cc \in \{=\}$  then
         $cc := next$ 
        call  $T(s; e_1)$ 
        if  $cc \in \{\oplus\}$  then
             $cc := next$ 
            call  $T(s; e_2)$ 
            if  $cc \in \{;\}$  then
                 $e := e_1 \cup e_2$ 
            return
    error // all syntactic error cases grouped here
end procedure

```

As an optimization, all the syntactic error cases are grouped at the end, and all checks on cc before each call to the procedure for T have been omitted as they are also performed inside T . The tree pointer is no longer necessary, as the semantic procedure integrates syntax analysis, thus it does not need to visit a pre-built syntax tree. A consequence is that the assigned variable v must be captured in the attribute d as soon as it is scanned in the input by means of the token cc (see the comment in the code).